

Pudzian: hierarchiczny agent Dueling Double DQN w grze battle-royale GUPB

Raport z projektu (Uczenie ze wzmocnieniem)

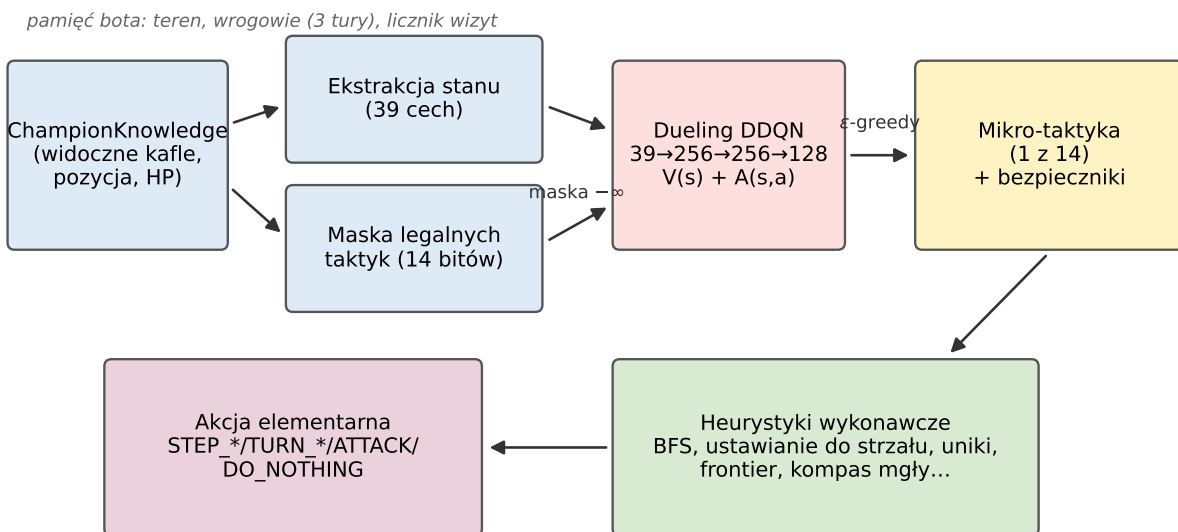
Lipiec 2026

1 Wybór akcji: architektura hierarchiczna

Pudzian gra w GUPB, turowej grze battle-royale na siatce 2D z obserwacją częściową: agent widzi tylko stożek pola widzenia w kierunku patrzenia, ma 8 HP, zbiera broń i mikstury, a kurcząca się mgła spycha graczy do menhiru. Punktacja zależy od kolejności śmierci, więc liczy się przede wszystkim przeżycie. Sterowanie odbywa się akcjami elementarnymi: kroki w 4 kierunkach (STEP_*), obroty (TURN_*), ATTACK i DO_NOTHING.

Kluczowa, niestandardowa decyzja projektowa: **sieć neuronowa nie wybiera akcji elementarnej**. Uczenie polityki na surowych krokach i obrotach przy częściowej obserwacji wymagałoby, aby DQN nauczył się jednocześnie nawigacji, geometrii zasięgu każdej broni i strategii. Zamiast tego rozdzielamy kompetencje:

- **DQN decyduje „co chcę osiągnąć”**: wybiera jedną z 14 *mikro-taktyk* (cel strategiczny na tę turę),
- **heurystyki decydują „jak to zrobić”**: deterministycznie tłumaczą taktykę na akcję elementarną (BFS do celu, obrót do strzału itd.).



Rysunek 1: Potok decyzyjny Pudziana: wiedza → stan 39D + maska → DQN → taktyka → heurystyki → akcja elementarna.

Dzięki temu sieć operuje w małej, semantycznej przestrzeni akcji, a sygnał uczący nie jest marnowany na naukę rzeczy algorytmicznie trywialnych (pathfinding). Epizod z perspektywy sieci jest też efektywnie krótszy: jedna decyzja taktyczna odpowiada sensownemu ruchowi, nie pojedynczemu obrotowi.

1.1 Przestrzeń akcji sieci: 14 mikro-taktyk

Grupa	Taktyka	Znaczenie
Walka	APPROACH	szarża na najbliższego wroga (nigdy nie cofa się)
	ALIGN_AXIS	ustawienie na pozycji strzału właściwej dla broni
	MAINTAIN_DIST	utrzymanie optymalnego dystansu broni (łuk: 5, miecz: 2...)
	DODGE_LEFT/RIGHT	unik prostopadle do osi bot-wróg
	FLEE_CLOSEST	ucieczka od najbliższego wroga
Przetrawianie	GET_POTION	marsz po najbliższą osiągalną miksturę
	GET_WEAPON	marsz po broń wyższego tieru
	HOLD_MENHIR	obrona pozycji przy menhirze (camping + atak intruzów)
	FOLLOW_MIST	ucieczka od mgły w stronę menhiru / bezpiecznej strefy
	EXPLORE	marsz ku nieodkrytym kratkom (frontier)
	SCAN	obrót w miejscu, budowa mapy
Timing	ATTACK	surowy cios <i>teraz</i> (naciąganie łuku, cios w zapamiętanego wroga, bait)
	WAIT	świadome stanie w miejscu (zasadzka)

Tabela 1: Mikro-taktyki. **ATTACK** i **WAIT** to jedyne taktyki mapowane 1:1 na akcję elementarną; sieć sama uczy się ich *timingu*, bo tego heurystyka nie umie ocenić.

1.2 Wektor stanu (39 cech)

Stan to ręcznie zaprojektowany, znormalizowany wektor: HP (/8), broń własna (one-hot 6), kierunek patrzenia (one-hot 4), najbliższy wróg (dystans/15, HP, tier broni, czy patrzy na nas, kierunek względny), najsłabszy widoczny wróg, menhir (znany?, dystans, kierunek), dystanse do mikstury i ulepszenia broni, mgła (czy stoję w niej, dystans, kompas od centroidu mgły, bliskość mgły do menhiru), liczba żywych, numer tury, odsetek odkrytej mapy oraz flaga naładowania łuku. Dystanse dzielone przez 15, aby sieć rozróżniała zasięg noża (1) od łuku (5).

1.3 Maskowanie akcji

Drugi niestandardowy element: w każdej turze liczona jest **maska legalnych taktyk**, a Q nielegalnych akcji ustawiane jest na $-\infty$ (maska trafia też do bufora doświadczeń i jest używana przy liczeniu celu Bellmana). Reguły:

- taktyki nawigacyjne walki tylko gdy wróg jest *osiągalny* BFS-em lub w zasięgu broni; wróg widoczny przez wodę nie odblokowuje **APPROACH**, co zapobiega oscylacjom przy braku ścieżki;
- **DODGE/FLEE** tylko gdy odpowiednie pole faktycznie jest przejezdne;
- **GET_POTION/GET_WEAPON/HOLD_MENHIR/FOLLOW_MIST** tylko gdy cel istnieje w pamięci;
- stojąc w **mgłę** legalne jest wyłącznie **FOLLOW_MIST**; przy $mgle \leq 3$ kratek dozwolone są tylko **APPROACH**, **ALIGN_AXIS** i **FOLLOW_MIST** (spójnie z bezpiecznikiem, patrz niżej);
- anty-degeneracje: blokada **SCAN** po 2 kolejnych skanach, zakaz naprzemiennych uników lewo-prawo, **WAIT** tylko przy widocznym wrogu.

Maskowanie radykalnie przycina przestrzeń eksploracji: losowa akcja podczas treningu jest zawsze *sensowna kontekstowo*, co czyni nawet w pełni losową politykę grywalną (wykorzystywane w fazie warmup).

1.4 Bezpieczniki (overrides)

Między wyborem sieci a wykonaniem działają trzy nadpisanie:

1. **bezpiecznik mgły**: gdy $mgła \leq 3$ kratek, taktyki inne niż ofensywne są zamieniane na **FOLLOW_MIST**; nagroda liczona jest za taktykę *faktycznie wykonaną*, więc uczenie pozostaje spójne;

2. **auto-atak:** przy taktykach nawigacyjnych (`APPROACH/ALIGN_AXIS/MAINTAIN_DIST`) bot atakuje natychmiast, jeśli wróg jest w zasięgu broni; sieć nie musi uczyć się oczywistości „bij, gdy możesz”;
3. **auto-leczenie:** przez 15 tur po zabiciu wroga, przy brakującym HP i widocznej miksturze, bot idzie ją wypić niezależnie od decyzji sieci.

1.5 Heurystyki wykonawcze

Warstwa heurystyk (`heuristics.py`) tłumaczy taktykę na akcję. Najważniejsze procedury:

Pathfinding (`find_path, move_towards`)

dwupoziomowy BFS: najpierw ścieżka *bezpieczna* (omijająca mgłę i ogień), a gdy takiej nie ma, ścieżka przez zagrożenia. Korzysta z pamięci terenu spoza pola widzenia; wyniki są memoizowane w obrębie tury.

Zasięg broni (`is_in_range`)

geometria trafienia per broń z uwzględnieniem przeszkód na linii strzału: bronie liniowe (nóż 1, miecz 3, łuk ∞ z LOS), topór w kształcie T, amulet na przekątnych $(\pm r, \pm r)$, $r \in \{1, 2\}$.

Ustawianie do strzału

trzy warianty zależne od broni: dla broni liniowych `align_to_axis` (wspólna oś X/Y), dla amuletu `align_to_diagonal` (przekątna), dla topora `align_to_adjacent` (dowolne z 8 pól przyległych). Pomocnicze `_optimal_turn` wybiera obrót o minimalnej liczbie tur.

Unik (`dodge_perpendicular`)

krok prostopadle do osi bot–wróg (schodzi z linii strzału łuku); przy zablokowanym kierunku próbuje przeciwnego.

Dystans (`maintain_distance`)

za blisko $\rightarrow$ odejdz, za daleko $\rightarrow$ podejdz, w zakresie $\rightarrow$ ustaw się do ataku; optymalny dystans zależny od broni.

Zasoby (`find_potion, find_weapon_upgrade`)

najbliższa *osiągalna* mikstura / broń o wyższym tierze; jeden BFS od pozycji bota zamiast BFS na kandydata. Licznik „utknięcia”: 5 tur samych obrotów $\rightarrow$ porzucenie celu i powrót do eksploracji.

Eksploracja (`find_frontier`)

wybiera osiągalną kratkę graniczną (znaną, z nieznanym sąsiadem), ważoną zyskiem informacyjnym, licznikiem odwiedzin (anty-oscyłacja) i kierunkiem na (szacowany) menhir.

Kompas mgły (`compute_mist_compass`)

znormalizowany wektor *od* centroidu widocznych kratek mgły; służy jako lokalny kierunek ucieczki i cecha stanu sieci.

Estymata menhiru (`estimate_menhir_from_mist`)

szacuje środek bezpiecznej strefy z krzywizny okręgu mgły; pozwala uciekać we właściwą stronę, zanim menhir zostanie zobaczony.

Czułość wroga (`enemy_facing_towards`)

czy wróg patrzy w naszą stronę (na osi dominującej różnicy współrzędnych); używane jako cecha stanu dla sieci.

Taniec amuletu

amulet oślepia (minimalne pole widzenia), więc przy `EXPLORE` z amuletem bot wykonuje maszynę stanów: podejdz do innej broni (zamiana upuszcza amulet) $\rightarrow$ 4 obroty skanujące $\rightarrow$ odejdz $\rightarrow$ wróć po amulet. Przerywana natychmiast, gdy pojawi się wróg.

Pamięć bota uzupełnia obserwację częściową: trwała mapa terenu (mgła utrwalana jako nieprzejezdna), krótkotrwała pamięć pozycji wrogów (*object permanence*, 3 tury) i licznik odwiedzin pól.

2 Uczenie

2.1 Sieć i algorytm

Dueling DQN: wspólny ekstraktor $39 \rightarrow 256 \rightarrow 256 \rightarrow 128$ (ReLU + LayerNorm) i dwie głowice: $V(s)$ oraz $A(s, a)$, agregowane jako $Q = V + A - \text{mean}_a A$. Uczenie metodą **Double DQN** (sieć online wybiera akcję, sieć docelowa ją wycenia) z celami **n-krokowymi** ($n = 3$):

$$y = R^{(n)} + \gamma^n Q_{\text{target}}(s', \arg \max_a Q_{\text{online}}(s', a)) (1 - d),$$

z maskowaniem nielegalnych akcji w $\arg \max$. Bufor doświadczeń to **PER** (SumTree, pojemność 1,5 mln) z wagami ważności ($\beta: 0,4 \rightarrow 1,0$). Strata Hubera, gradient clipping (1,0), miękka aktualizacja sieci docelowej ($\tau = 0,005$), Adam ($3 \cdot 10^{-4}$), batch 256. Wysokie $\gamma = 0,997$ jest celowe: sygnał terminalny musi propagować się przez ~ 150 tur epizodu ($0,99^{150} \approx 0,22$, ale $0,997^{150} \approx 0,64$).

2.2 Funkcja nagrody

Brak nagrody bazowej za przeżycie (zapobiega wyuczonej pasywności), nagradzane są *działania*:

Zdarzenie	Nagroda	Zdarzenie	Nagroda
zabicie wroga (max 2/turę)	+10	tura w mgłę / w ogniu	-1,5 / -3,0
atak na wroga w zasięgu	+1,0	otrzymane obrażenia	-1,5/HP
ulepszenie broni ($\times$ skok tieru)	+2,0	ignorowanie ulepszenia broni	-0,3
wyleczenie	+0,75/HP	spam SCAN	do -0,65
skuteczny unik przy wrogu	+0,1	MAINTAIN_DIST z nożem/zwojem	-0,2
nowa odkryta kratka	+0,03 (0,01 po mgłę)	trzymanie menhiru przy mgłę	+0,15
nagroda terminalna z <code>praise()</code> : liniowo od miejsca, $\approx \pm 30$			

Tabela 2: Kształtowanie nagrody. Nagroda za zabójstwo wymaga, by wróg był faktycznie w zasięgu w momencie ataku (eliminacja fałszywego kredytu, gdy wroga zabił kto inny); pierwszy strzał łukiem (naciągnięcie, 0 obrażeń) nie jest nagradzany.

3 Trening

3.1 Ape-X: aktorzy + learner

Trening w stylu **Ape-X**: 14 procesów-aktorów (CPU, tylko inferencja) gra równolegle pełne mecze i wysyła przejścia n-krokowe kolejką do jednego procesu-learnera (GPU), który trenuje sieć i co 5 s rozgłasza świeże wagi. Każdy aktor ma **stały, indywidualny epsilon** $\varepsilon_i = 0,4^{1+3i/(N-1)}$ (od 0,40 do 0,026; rys. 4 po lewej). Zamiast globalnego wygaszania eksploracja jest więc zdywersyfikowana między aktorami przez cały trening. Pierwsze 30 tys. przejść to *warmup* z $\varepsilon = 1$: dzięki masce akcji nawet losowa polityka gra sensownie i zapełnia bufor PER zróżnicowanymi epizodami. Learner utrzymuje stały *replay ratio* 4 aktualizację gradientu na zebrane przejście.

3.2 Curriculum i liga self-play

Trudność rośnie w 4 poziomach; awans po osiągnięciu progu top-3 w oknie 50 gier *prawie zachłannych* aktorów ($\varepsilon \leq 0,1$):

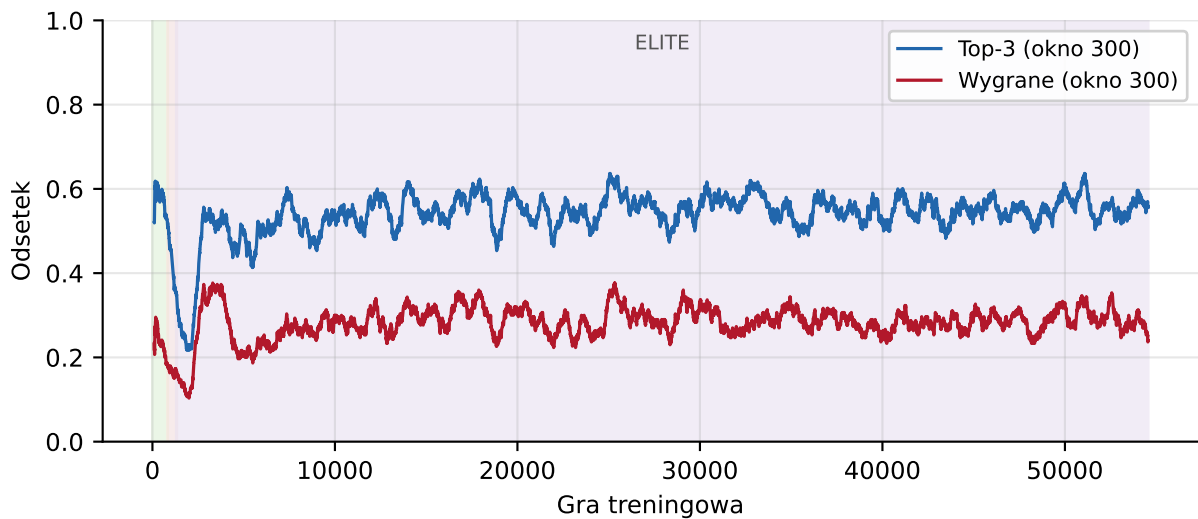
Poziom	Przeciwnicy	Odsetek botów skryptowych	Areny	Próg top-3
ROOKIE	4	0 %	1	65 %
AMATEUR	7	25 %	3	38 %
VETERAN	10	55 %	4	25 %
ELITE	10	80 %	6	brak

Tabela 3: Curriculum. Resztę przeciwników stanowią boty losowe; wśród skryptowych zawsze grają dwa najsilniejsze. Na wyższych poziomach 30 % miejsc „inteligentnych” może zająć **dawny Pudzian z ligi self-play**: co godzinę zapisywany jest snapshot sieci (utrzymywanych 10 ostatnich), z którym bot gra przeciw własnym wcześniejszym wersjom.

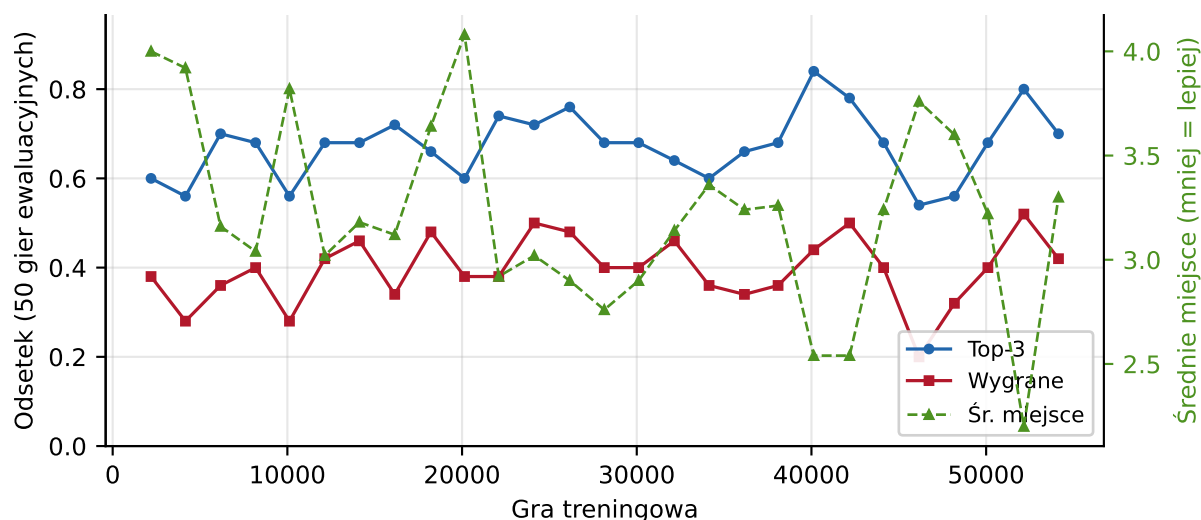
Co 500 gier osobny proces ewaluje bieżącą sieć zachłannie ($\varepsilon = 0$) na 50 grach (10 przeciwników, 50 % botów); najlepszy wynik top-3 wyznacza checkpoint `pudzian_dqn_best.pt`.

4 Wyniki

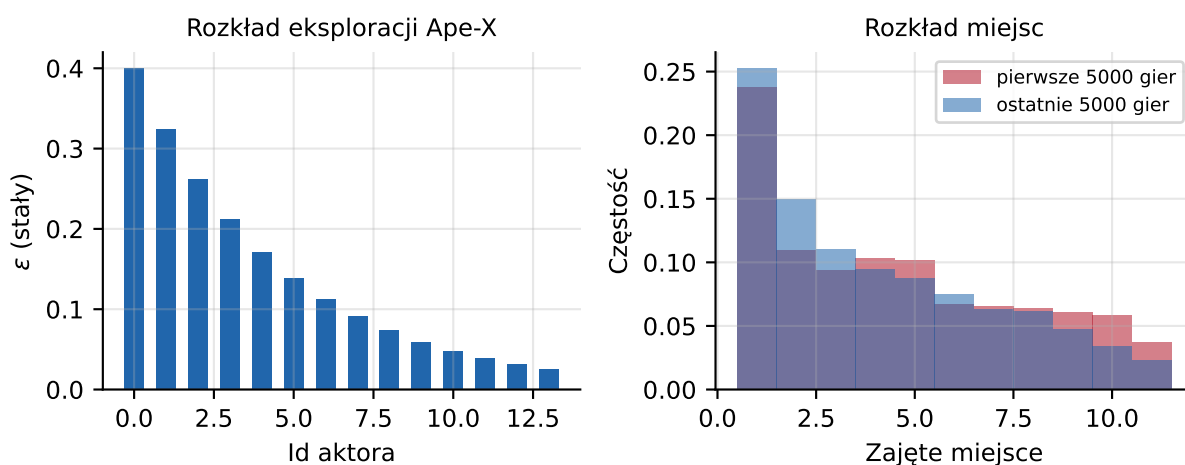
Przeprowadzony trening objął **54 562 gry** (~ 8 mln przejść). Curriculum zostało pokonane szybko (ROOKIE ~ 830 gier, VETERAN ~ 480), więc 98 % treningu odbyło się na poziomie ELITE, czyli najtrudniejszym ustawieniu.



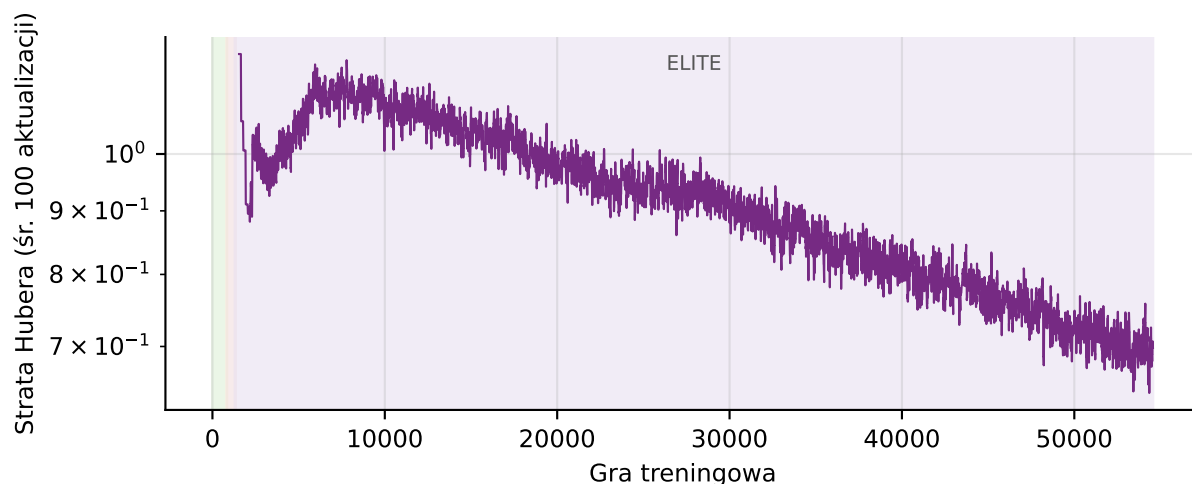
Rysunek 2: Krocząca skuteczność aktorów prawie zachłannych ($\varepsilon \leq 0,1$) podczas treningu; tła oznaczają poziom curriculum. Na ELITE odsetek top-3 rośnie z $\sim 35\%$ do $\sim 55\%$, a wygranych do $\sim 25\%$.



Rysunek 3: Ewaluacja zachłanna (50 gier co ~ 2000 gier treningowych). Najlepszy model osiąga **84 % top-3** (gra 40 132); końcowo 70 % top-3, 42 % wygranych, średnie miejsce 3,3/11.



Rysunek 4: Po lewej: stałe epsilon 14 aktorów Ape-X (skala geometryczna 0,40 $\rightarrow$ 0,026). Po prawej: rozkład zajmowanych miejsc, w którym masa przesuwa się z miejsc 5–8 ku 1–3 (średnio 4,57 $\rightarrow$ 4,11 mimo trudniejszych przeciwników na ELITE).



Rysunek 5: Strata Hubera (średnia z 100 aktualizacji, skala logarytmiczna). Skoki korelują ze zmianami poziomu trudności i pojawianiem się nowych przeciwników z ligi; brak dywergencji przy $\gamma = 0,997$.

5 Wnioski

- Hierarchia *DQN wybiera cel, heurystyki wykonują* okazała się kluczowa: mała semantyczna przestrzeń akcji + maskowanie sprawiają, że eksploracja jest tania, a sieć uczy się wyłącznie decyzji, których nie da się zapisać regułą (kiedy walczyć, kiedy uciekać, timing ataku).
- Największy wpływ na stabilność miały poprawki *anty-degeneracyjne*: gating nagrody za zabójstwo warunkiem zasięgu, brak nagrody za samo przeżycie, kary za spam **SCAN** i spójność maski z bezpiecznikiem mgły (bez niej Q nadpisywanych akcji nigdy się nie aktualizowało).
- Stałe, zróżnicowane epsilony Ape-X uprościły trening (brak strojenia harmonogramu wygaszania) i dostarczały świeżej eksploracji do końca, co widać w ciągłym wzroście top-3 na ELITE.
- Bot osiąga 84 % top-3 i ~40 % wygranych przeciw mieszance botów skryptowych; dalszy zysk wymagałby prawdopodobnie bogatszej reprezentacji stanu (surowa mapa + konwolucje) zamiast 39 ręcznych cech.